```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
```

```
data = pd.read_csv('/content/sample_data/Churn_Modelling.csv')

data.head(4)
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenur
0	1	15634602	Hargrave	619	France	Female	42	
1	2	15647311	Hill	608	Spain	Female	41	
2	3	15619304	Onio	502	France	Female	42	
3	4	15701354	Boni	699	France	Female	39	

Próximas etapas: Gerar código com `data` New interactive sheet

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   RowNumber        10000 non-null  int64
 1   CustomerId       10000 non-null  int64
 2   Surname          10000 non-null  object
 3   CreditScore      10000 non-null  int64
 4   Geography        10000 non-null  object
 5   Gender           10000 non-null  object
 6   Age              10000 non-null  int64
 7   Tenure           10000 non-null  int64
 8   Balance          10000 non-null  float64
 9   NumOfProducts    10000 non-null  int64
 10  HasCrCard        10000 non-null  int64
 11  IsActiveMember   10000 non-null  int64
 12  EstimatedSalary  10000 non-null  float64
 13  Exited           10000 non-null  int64
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
```

```
data.describe()
```

	RowNumber	CustomerId	CreditScore	Age	Tenure	
count	10000.00000	1.000000e+04	10000.000000	10000.000000	10000.000000	10
mean	5000.50000	1.569094e+07	650.528800	38.921800	5.012800	76
std	2886.89568	7.193619e+04	96.653299	10.487806	2.892174	62
min	1.00000	1.556570e+07	350.000000	18.000000	0.000000	
25%	2500.75000	1.562853e+07	584.000000	32.000000	3.000000	
50%	5000.50000	1.569074e+07	652.000000	37.000000	5.000000	97
75%	7500.25000	1.575323e+07	718.000000	44.000000	7.000000	127
max	10000.00000	1.581569e+07	850.000000	92.000000	10.000000	250

```
data.tail()
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	T
9995	9996	15606229	Obijiaku	771	France	Male	39	
9996	9997	15569892	Johnstone	516	France	Male	35	
9997	9998	15584532	Liu	709	France	Female	36	
9998	9999	15682355	Sabbatini	772	Germany	Male	42	
9999	10000	15628319	Walker	792	France	Female	28	

```
data.isnull().sum()
```

	0
RowNumber	0
CustomerId	0
Surname	0
CreditScore	0
Geography	0
Gender	0
Age	0
Tenure	0
Balance	0
NumOfProducts	0
HasCrCard	0
IsActiveMember	0
EstimatedSalary	0
Exited	0

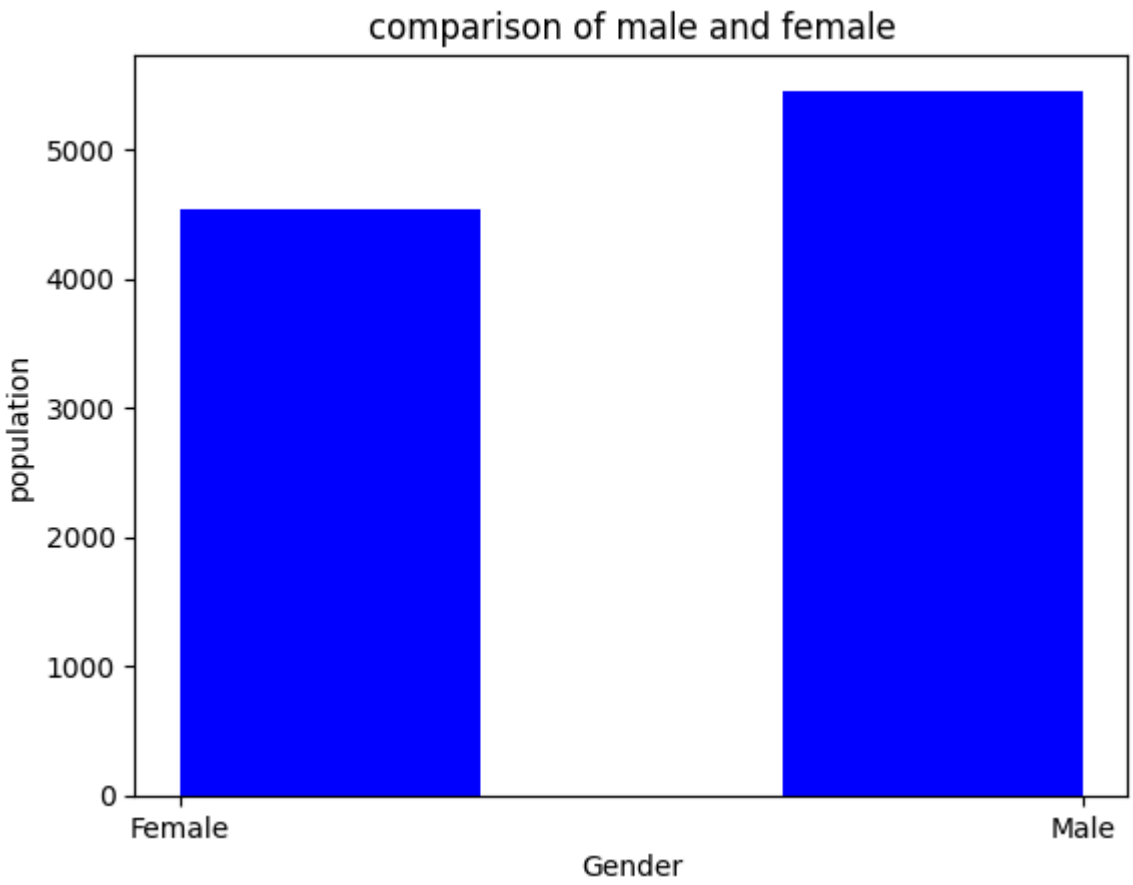
dtype: int64

```
data['Gender'].value_counts()
```

Gender	count
Male	5457
Female	4543

dtype: int64

```
plt.hist(x = data.Gender, bins = 3, color = 'blue')
plt.title('comparison of male and female')
plt.xlabel('Gender')
plt.ylabel('population')
plt.show()
```



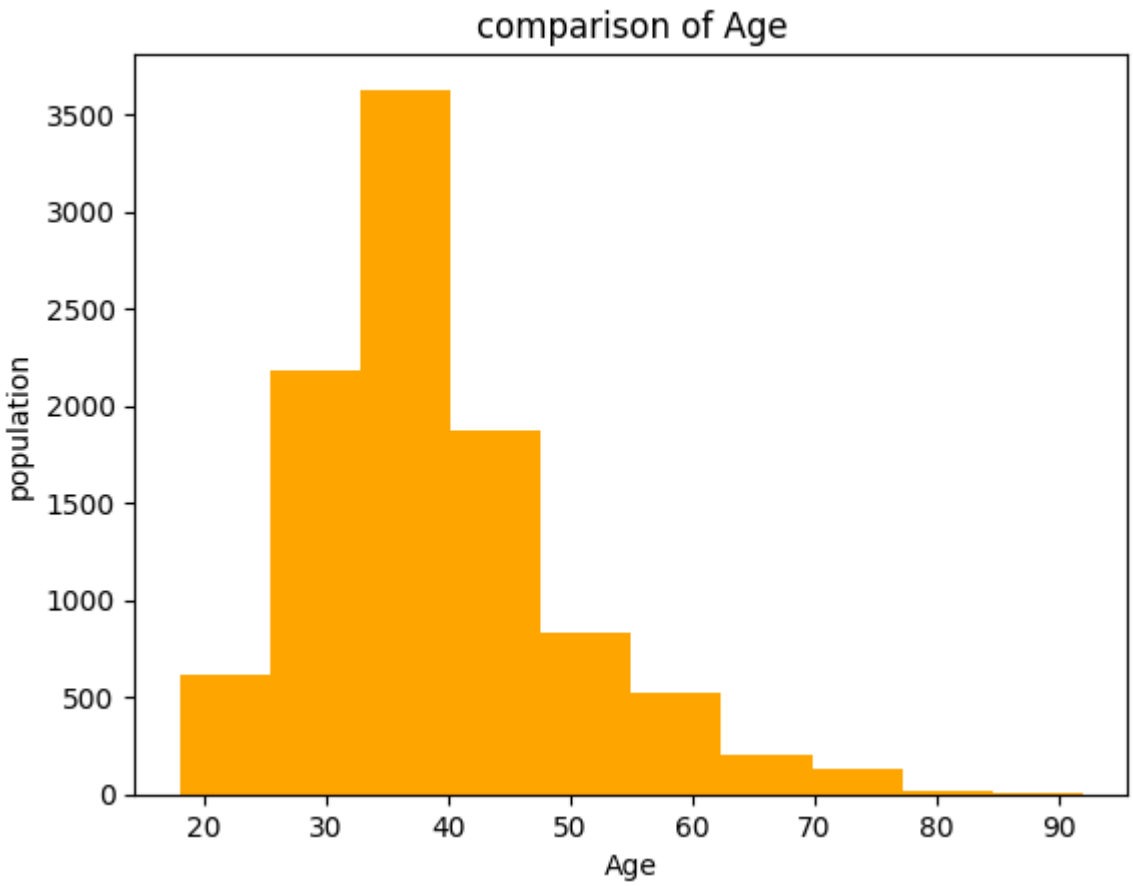

```
data['Age'].value_counts()
```

Age	count
37	478
38	477
35	474
36	456
34	447
...	...
84	2
82	1
88	1
85	1
83	1

70 rows × 1 columns

dtype: int64

```
plt.hist(x = data.Age, bins = 10, color = 'orange')
plt.title('comparison of Age')
plt.xlabel('Age')
plt.ylabel('population')
plt.show()
```

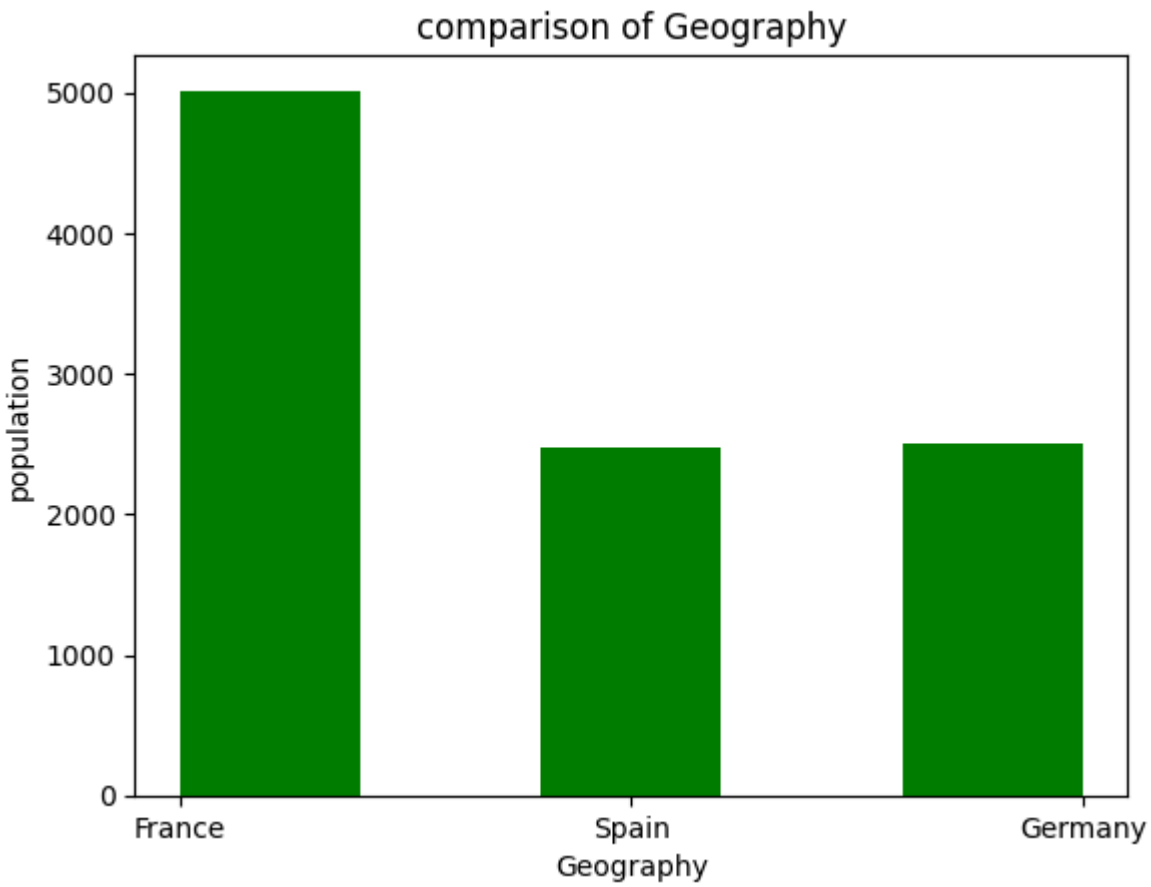



```
data['Geography'].value_counts()
```

Geography	count
France	5014
Germany	2509
Spain	2477

dtype: int64

```
plt.hist(x = data.Geography, bins = 5, color = 'green')
plt.title('comparison of Geography')
plt.xlabel('Geography')
plt.ylabel('population')
plt.show()
```

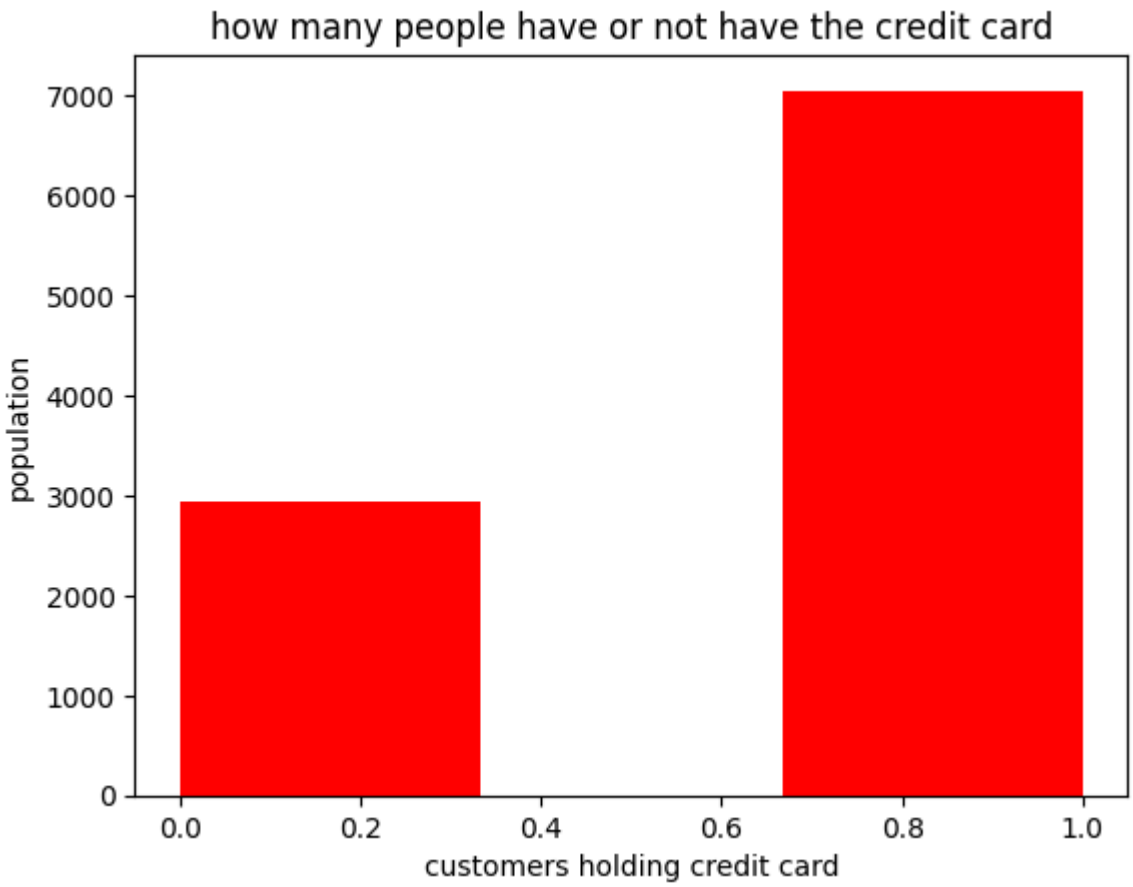



```
data['HasCrCard'].value_counts()
```

HasCrCard	count
1	7055
0	2945

dtype: int64

```
plt.hist(x = data.HasCrCard, bins = 3, color = 'red')
plt.title('how many people have or not have the credit card')
plt.xlabel('customers holding credit card')
plt.ylabel('population')
plt.show()
```



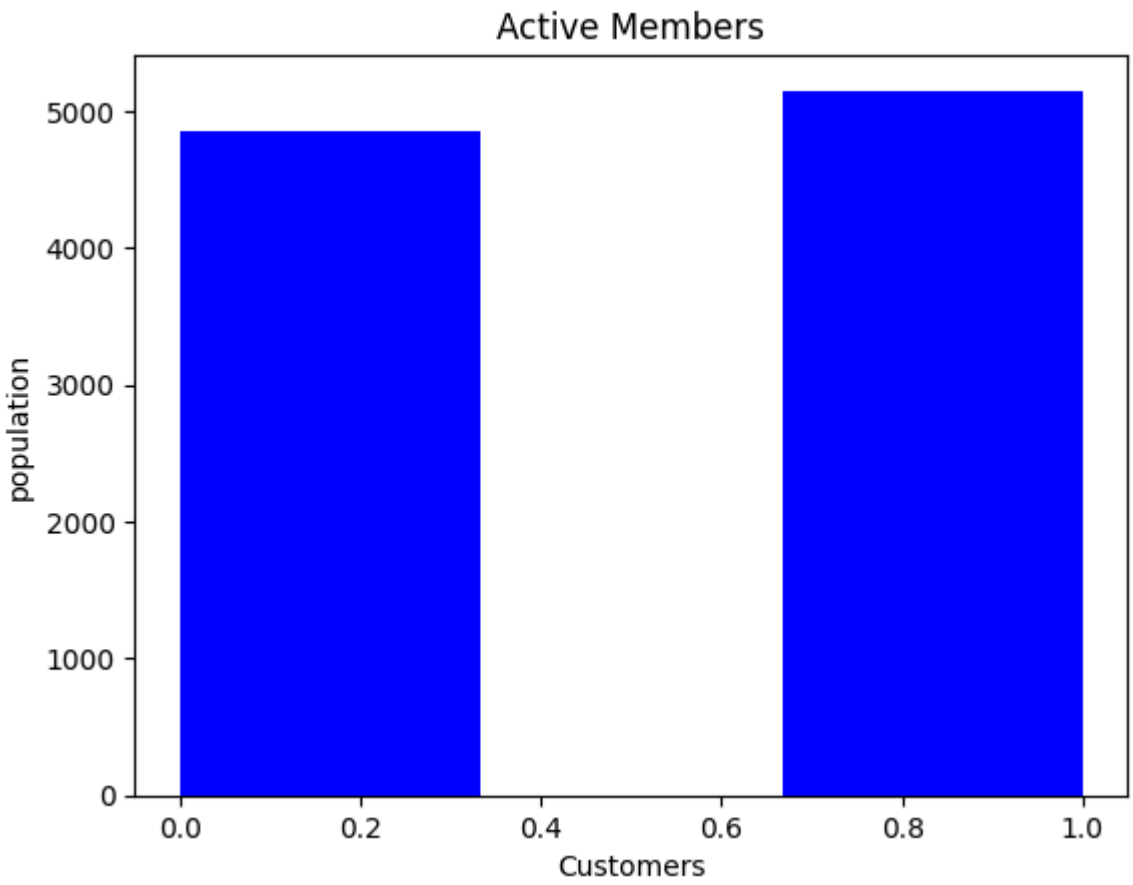
Comece a programar ou gere código com IA.

```
data['IsActiveMember'].value_counts()
```

IsActiveMember	count
1	5151
0	4849

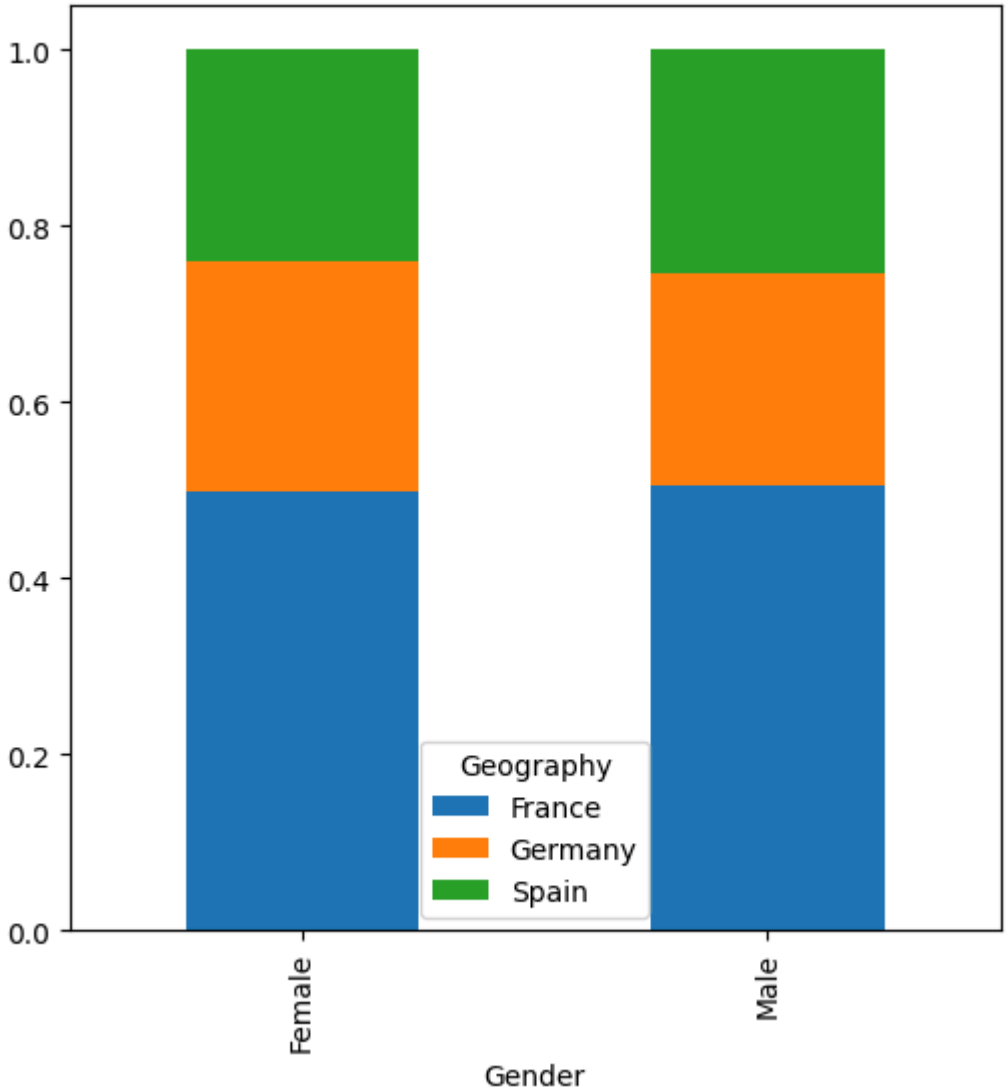
dtype: int64

```
plt.hist(x = data.IsActiveMember, bins = 3, color = 'blue')
plt.title('Active Members')
plt.xlabel('Customers')
plt.ylabel('population')
plt.show()
```



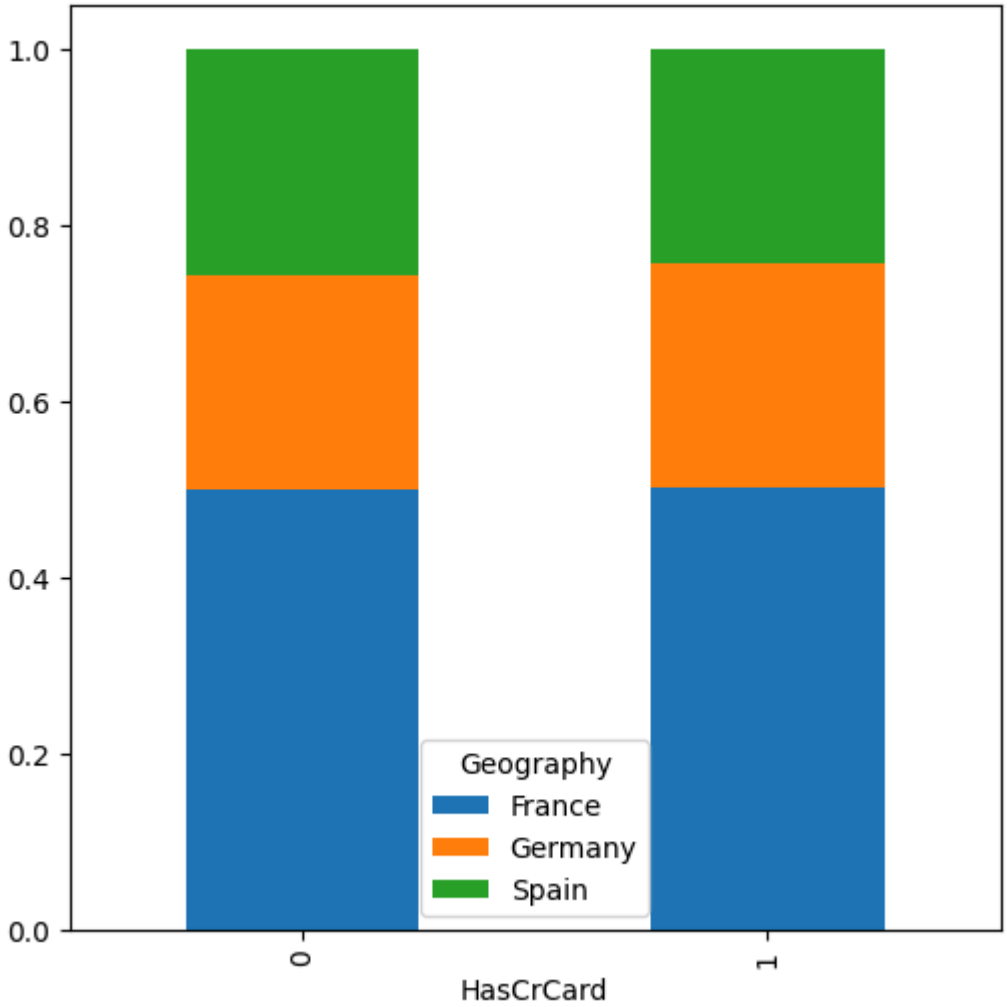

```
Gender = pd.crosstab(data['Gender'],data['Geography'])
Gender.div(Gender.sum(1).astype(float), axis=0).plot(kind="bar", stacked=Tru
```

```
<Axes: xlabel='Gender'>
```



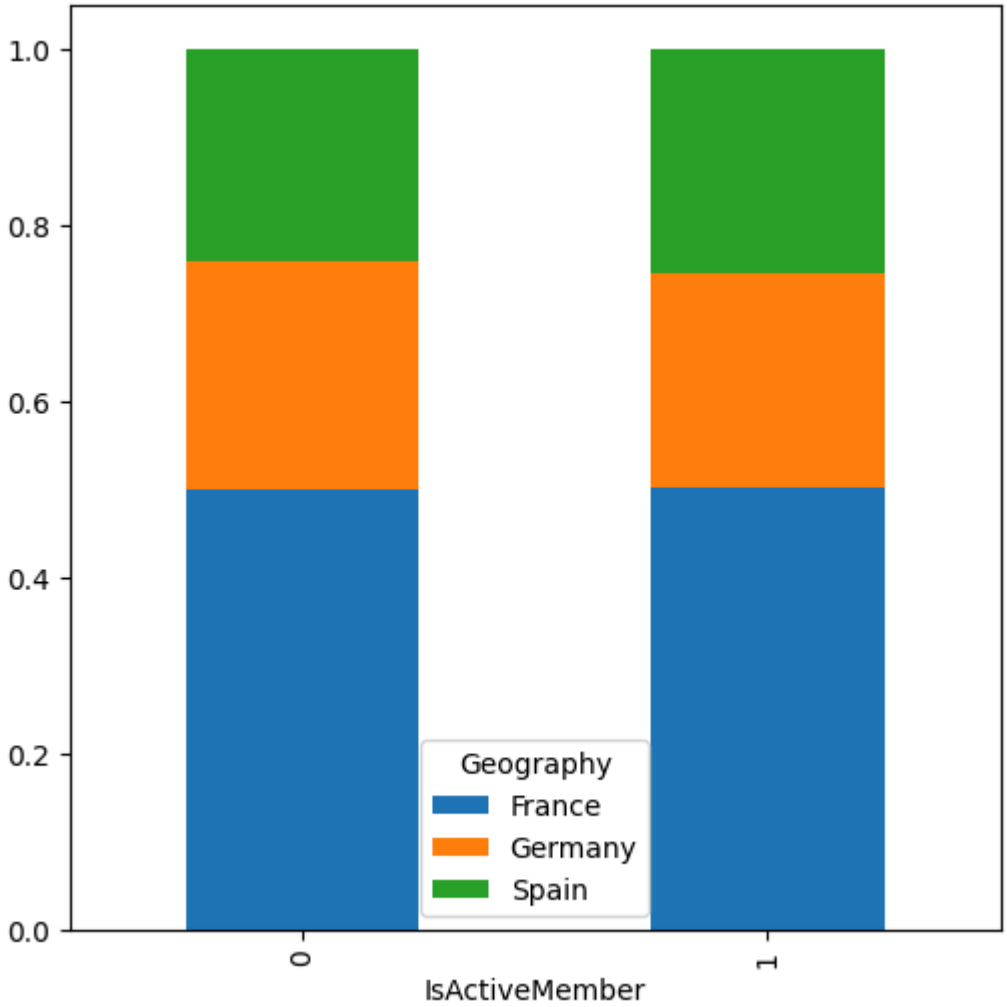

```
rCard = pd.crosstab(data['HasCrCard'], data['Geography'])
rCard.div(HasCrCard.sum(1).astype(float), axis = 0).plot(kind = 'bar',
                                                         stacked = True,figsize = (6, 6))
```

```
<Axes: xlabel='HasCrCard'>
```



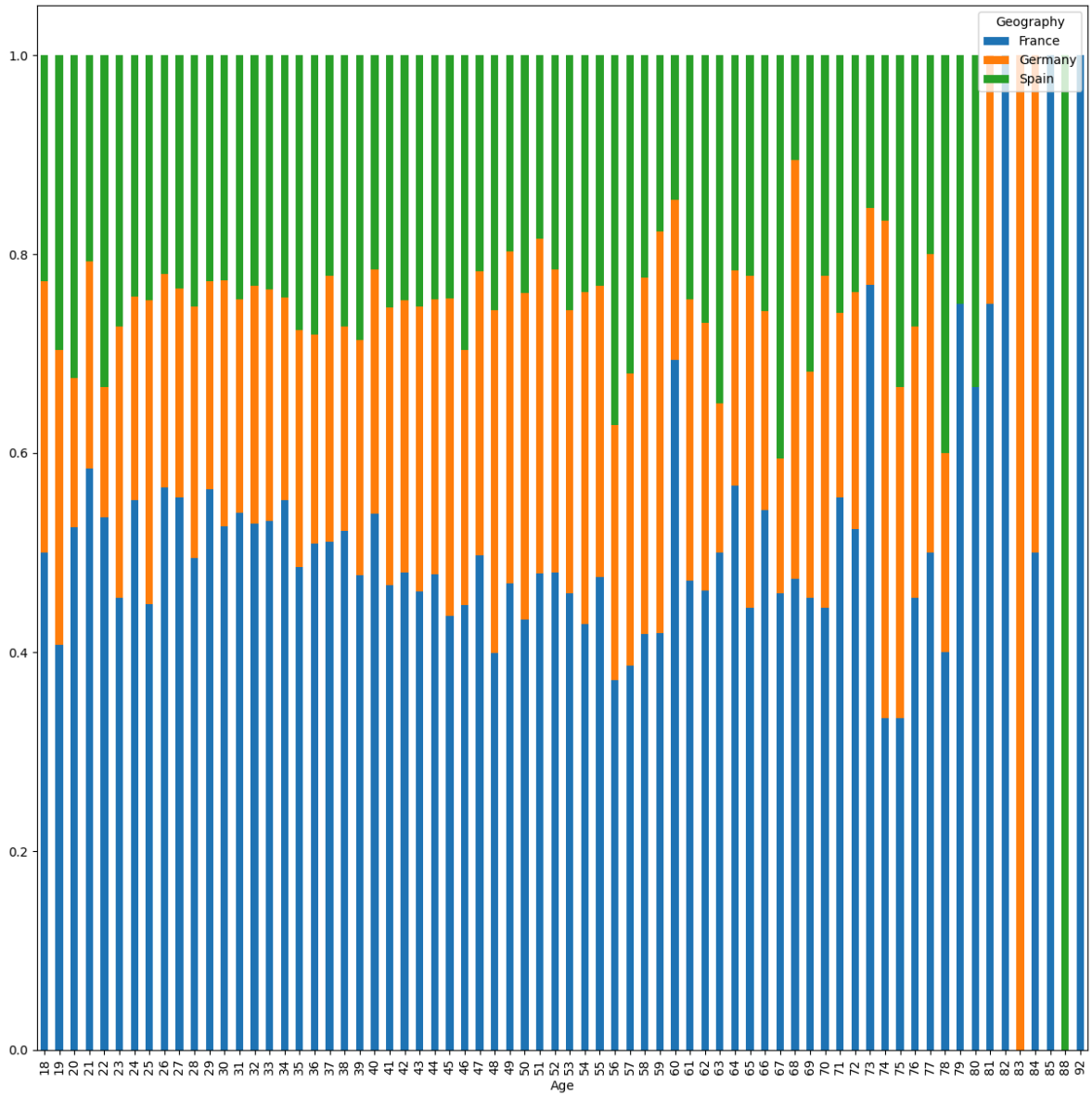

```
:iveMember = pd.crosstab(data['IsActiveMember'], data['Geography'])
:iveMember.div(IsActiveMember.sum(1).astype(float), axis = 0).plot(kind = 'ba
                                                stacked = True, figsize= (6, 6))
```

```
<Axes: xlabel='IsActiveMember'>
```




```
: pd.crosstab(data['Age'], data['Geography'])
!iv(Age.sum(1).astype(float), axis = 0).plot(kind = 'bar',
                                             stacked = True, figsize = (15,15))
```

```
<Axes: xlabel='Age'>
```

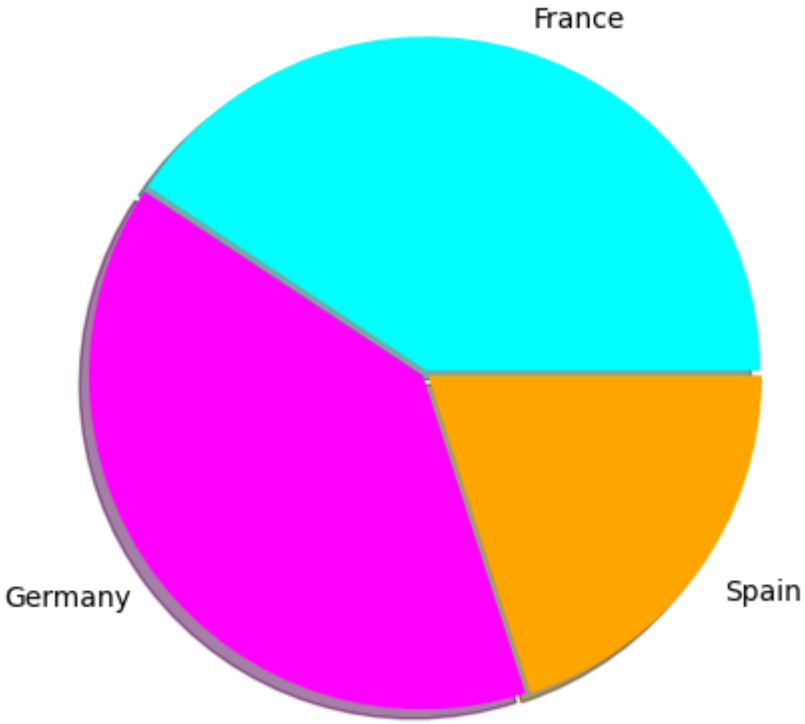



```
total_france = data.Balance[data.Geography == 'France'].sum()
total_germany = data.Balance[data.Geography == 'Germany'].sum()
total_spain = data.Balance[data.Geography == 'Spain'].sum()

print("Total Balance in France :",total_france)
print("Total Balance in Germany :",total_germany)
print("Total Balance in Spain :",total_spain)
```

```
Total Balance in France : 311332479.49
Total Balance in Germany : 300402861.38
Total Balance in Spain : 153123552.01
```

```
labels = 'France', 'Germany', 'Spain'
colors = ['cyan', 'magenta', 'orange']
sizes =  [311, 300, 153]
explode = [ 0.01, 0.01, 0.01]

plt.pie(sizes, colors = colors, labels = labels, explode = explode, shadow =
```

```
plt.axis('equal')
plt.show()
```




```
data = data.drop(['CustomerId', 'Surname', 'RowNumber'], axis = 1)



print(data.columns)
```

```
Index(['CreditScore', 'Geography', 'Gender', 'Age', 'Tenure', 'Balance',
       'NumOfProducts', 'HasCrCard', 'IsActiveMember', 'EstimatedSalary',
       'Exited'],
      dtype='object')
```

```
data.shape
```

```
(10000, 11)
```

```
x = data.iloc[:,0:10]
y = data.iloc[:,10]

print(x.shape)
print(y.shape)

print(x.columns)
#print(y)
```

```
(10000, 10)
(10000,)
Index(['CreditScore', 'Geography', 'Gender', 'Age', 'Tenure', 'Balance',
```

```
       'NumOfProducts', 'HasCrCard', 'IsActiveMember', 'EstimatedSalary'],
      dtype='object')
```

```
x = pd.get_dummies(x)

x.head()
```

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMemb
0	619	42	2	0.00	1	1	
1	608	41	1	83807.86	1	0	
2	502	42	8	159660.80	3	1	
3	699	39	1	0.00	2	0	
4	850	43	2	125510.82	1	1	

Próximas etapas: Gerar código com x New interactive sheet

```
x.shape
```

```
(10000, 13)
```

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.25,

print(x_train.shape)
print(y_train.shape)
print(x_test.shape)
print(y_test.shape)
```

```
(7500, 13)
(7500,)
(2500, 13)
(2500,)
```

```
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
x_train = sc.fit_transform(x_train)
x_test = sc.fit_transform(x_test)

x_train = pd.DataFrame(x_train)
x_train.head()
```

	0	1	2	3	4	5	6	
0	-0.735507	0.015266	0.008860	0.673160	2.535034	-1.553624	-1.034460	-1.64
1	1.024427	-0.652609	0.008860	-1.207724	0.804242	0.643657	-1.034460	-0.07
2	0.808295	-0.461788	1.393293	-0.356937	0.804242	0.643657	0.966688	-0.99
3	0.396614	-0.080145	0.008860	-0.009356	-0.926551	0.643657	0.966688	-1.59
4	-0.467915	1.255605	0.701077	-1.207724	0.804242	0.643657	0.966688	1.28

Próximas etapas: Gerar código com `x_train` New interactive sheet

```
#modelagem tipo - Decision Tree

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix

model = DecisionTreeClassifier()
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

print("Training Accuracy :", model.score(x_train, y_train))
print("Testing Accuaracy :", model.score(x_test, y_test))

cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
Training Accuracy : 1.0
Testing Accuaracy : 0.8024
[[1708  283]
 [ 211  298]]
```

```
#modelagem tipo - Random Forest

from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

print("Training Accuracy :", model.score(x_train, y_train))
print("Testing Accuracy :", model.score(x_test, y_test))

cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
Training Accuracy : 1.0
Testing Accuracy : 0.8672
```

```
[[1911   80]
 [ 252  257]]
```

```
# k fold cross validatio

from sklearn.model_selection import cross_val_score

cvs = cross_val_score(estimator = model, X = x_train, y = y_train, cv = 10)
print(cvs)
```

```
[0.86533333 0.856      0.86133333 0.86133333 0.85466667 0.85333333
 0.86266667 0.856      0.85333333 0.85466667]
```

```
print("Mean Accuracy :", cvs.mean())
print("Variance :", cvs.std())
```

```
Mean Accuracy : 0.8578666666666667
Variance : 0.004139779919002646
```

```
#modelagem tipo - Logistic Regression

from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

print("Training Accuracy :", model.score(x_train, y_train))
print("Testing Accuracy :", model.score(x_test, y_test))

cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
Training Accuracy : 0.8096
Testing Accuracy : 0.8092
[[1916   75]
 [ 402  107]]
```

```
#Support Vector Machine

from sklearn.svm import SVC

model = SVC()
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

print("Training Accuracy :", model.score(x_train, y_train))
print("Testing Accuracy :", model.score(x_test, y_test))

cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
Training Accuracy : 0.8625333333333334
Testing Accuracy : 0.8616
[[1951   40]
 [ 306  203]]
```

```
# k fold cross validatio

from sklearn.model_selection import cross_val_score

cvs = cross_val_score(estimator = model, X = x_train, y = y_train, cv = 10)
print(cvs)
```

```
[0.864      0.852      0.864      0.85733333 0.84266667 0.844
 0.852      0.85333333 0.84533333 0.85066667]
```

```
print("Mean Accuracy :", cvs.mean())
print("Variance :", cvs.std())
```

```
Mean Accuracy : 0.8525333333333333
Variance : 0.007160384843785353
```

```
#Multi Layer Perceptron

from sklearn.neural_network import MLPClassifier

model = MLPClassifier(hidden_layer_sizes = (100, 100), activation ='relu',
                      solver = 'adam', max_iter = 50)
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

print("Training Accuracy :", model.score(x_train, y_train))
print("Testing Accuracy :", model.score(x_test, y_test))

cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
Training Accuracy : 0.8849333333333333
Testing Accuracy : 0.8544
[[1861  130]
 [ 234  275]]
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_pe
  warnings.warn(
```

```
#Artificial Neural Networks

import keras
from keras.models import Sequential
from keras.layers import Dense
```

```
# creating the model
```

```
# Creating the model
model = Sequential()

# first hidden layer
model.add(Dense(units = 8, kernel_initializer = 'uniform', activation = 'rel

# second hidden layer
model.add(Dense(units = 8, kernel_initializer = 'uniform', activation = 'rel

# third hidden layer
model.add(Dense(units = 8, kernel_initializer = 'uniform', activation = 'rel

# fourth hidden layer
model.add(Dense(units = 8, kernel_initializer = 'uniform', activation = 'rel

# fifth hidden layer
model.add(Dense(units = 8, kernel_initializer = 'uniform', activation = 'rel

# output layer
model.add(Dense(units = 1, kernel_initializer = 'uniform', activation = 'sig

# Compiling the NN
# binary_crossentropy loss function used when a binary output is expected
model.compile(optimizer = 'adam', loss = 'binary_crossentropy', metrics = ['

model.fit(x_train, y_train, batch_size = 10, epochs = 50)
```

```
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8352 - loss: 0.4019
Epoch 39/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8355 - loss: 0.4010
Epoch 40/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8343 - loss: 0.4013
Epoch 41/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8348 - loss: 0.4003
Epoch 42/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8365 - loss: 0.3998
Epoch 43/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8331 - loss: 0.3988
Epoch 44/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8360 - loss: 0.4001
Epoch 45/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8351 - loss: 0.3998
Epoch 46/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8347 - loss: 0.3994
Epoch 47/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8348 - loss: 0.3996
Epoch 48/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8345 - loss: 0.3997
Epoch 49/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8368 - loss: 0.3988
Epoch 50/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8360 - loss: 0.3991
<keras.src.callbacks.history.History at 0x7fd3387538c0>
```

```
# creating the model
model = Sequential()

from keras.layers import Dropout

# first hidden layer
model.add(Dense(units = 8, kernel_initializer = 'uniform', activation = 'rel
model.add(Dropout(0.5))

# second hidden layer
model.add(Dense(units = 8, kernel_initializer = 'uniform', activation = 'rel
model.add(Dropout(0.5))

# output layer
model.add(Dense(units = 1, kernel_initializer = 'uniform', activation = 'sig

# Compiling the NN
# binary_crossentropy loss function used when a binary output is expected
model.compile(optimizer = 'adam', loss = 'binary_crossentropy', metrics = ['

model.fit(x_train, y_train, batch_size = 10, epochs = 50)
```

```
Epoch 27/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4513
Epoch 28/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4531
Epoch 29/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4527
Epoch 30/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4524
Epoch 31/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.7963 - loss: 0.4510
Epoch 32/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4565
Epoch 33/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4546
Epoch 34/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4535
Epoch 35/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4495
Epoch 36/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4541
Epoch 37/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4491
Epoch 38/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4517
Epoch 39/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4470
Epoch 40/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.7963 - loss: 0.4467
Epoch 41/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4518
Epoch 42/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4532
Epoch 43/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4544
Epoch 44/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4522
Epoch 45/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4548
Epoch 46/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4539
Epoch 47/50
750/750 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7963 - loss: 0.4494
```